

Register Number: 192425223

Name: M Venkata Sai

CO1 – AT3 MLA 0201 – Fundamentals Of Machine Learning

Annexure A

Experiment 1: Probability Theory (20 Marks)

Dataset: Breast Cancer Wisconsin Diagnostic Dataset

Task:

Using the **Breast Cancer Wisconsin Diagnostic Dataset**, calculate the probability of each class (**Malignant** and **Benign**) and use the computed probabilities to predict the class of a new data instance.

Experiment 2: Bayes Decision Theory (15 Marks)

Dataset: SMS Spam Collection Dataset

Task:

Using the **SMS Spam Collection Dataset**, implement Bayes theorem to calculate the posterior probability for a given message and classify it as **Spam** or **Ham (Not Spam)**.

Experiment 3: Information Theory (15 Marks)

Dataset: Play Tennis Dataset

Task:

Using the **Play Tennis Dataset**, calculate the entropy and information gain for all input attributes and identify the attribute with the highest information gain.

1. Probability Theory

Aim:

To calculate the probability of each class (Malignant and Benign) using the Breast Cancer Wisconsin Diagnostic Dataset and predict the class of a new data instance using probability-based classification.

Program:

```
import pandas as pd from google.colab import
files
from sklearn.model_selection import train_test_split from
sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report uploaded =
files.upload()
file_name = list(uploaded.keys())[0] data =
pd.read_csv(file_name) print(data.head())
data = data.drop("id", axis=1)
data = data.drop("Unnamed: 32", axis=1) print("\nClass Count:")
print(data["diagnosis"].value_counts())
probability = data["diagnosis"].value_counts(normalize=True)
print("\nProbability of Each Class:") print(probability)
data["diagnosis"] = data["diagnosis"].map({"B": 0, "M": 1}) x =
data.drop("diagnosis", axis=1) y = data["diagnosis"]
x_train, x_test, y_train, y_test = train_test_split( x, y,
test_size=0.2, random_state=42
model = GaussianNB()
model.fit(x_train, y_train) y_pred =
model.predict(x_test)
print("\nAccuracy:")
print(accuracy_score(y_test, y_pred)) print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred)) print("\nClassification Report:")
print(classification_report(y_test, y_pred)) new_instance = x_test.iloc[[0]] prediction.
```

Output:

```
Class Count:
diagnosis
B    357
M    212
Name: count, dtype: int64

Probability of Each Class:
diagnosis
B    0.627417
M    0.372583
Name: proportion, dtype: float64

Accuracy:
0.9736842105263158

Confusion Matrix:
[[71  0]
 [ 3 40]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.96	1.00	0.98	71
1	1.00	0.93	0.96	43
accuracy			0.97	114
macro avg	0.98	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

```
Predicted Class: Benign
```

Result:

The probability of Malignant and Benign classes was successfully calculated using the Breast Cancer Wisconsin Diagnostic Dataset. The experiment demonstrates the application of probability theory in medical disease prediction.

2. Bayes Decision Theory

Aim:

To implement Bayes theorem using the SMS Spam Collection Dataset and calculate the posterior probability for a given message to classify it as Spam or Ham (Not Spam).

Program:

```
import pandas as pd from
google.colab import files
from sklearn.model_selection import train_test_split from
sklearn.feature_extraction.text import CountVectorizer from
sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report uploaded =
files.upload()
file_name = list(uploaded.keys())[0]
data = pd.read_csv(file_name, encoding="latin-1") data =
data.iloc[:, :2]
data.columns = ["class", "sms"] data =
data.dropna() print(data.head())
print("\nClass Distribution:")
print(data["class"].value_counts()) print("\nClass
Probability:")
print(data["class"].value_counts(normalize=True)) x
= data["sms"] y = data["class"]
x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=0.2, random_state=42) vectorizer =
CountVectorizer()
x_train = vectorizer.fit_transform(x_train) x_test = vectorizer.transform(x_test) model =
MultinomialNB()
```

Output:

```
archive (8).zip(application/x-zip-compressed) - 215934 bytes, last modified: 7/29/2026 - 100% done
Saving archive (8).zip to archive (8).zip
class sms
0 ham Go until jurong point, crazy.. Available only ...
1 ham Ok lar... Joking wif u oni...
2 spam Free entry in 2 a wkly comp to win FA Cup fina...
3 ham U dun say so early hor... U c already then say...
4 ham Nah I don't think he goes to usf, he lives aro...

Class Distribution:
class
ham 4825
spam 747
Name: count, dtype: int64

Class Probability:
class
ham 0.865937
spam 0.134063
Name: proportion, dtype: float64

Accuracy:
0.9838565022421525

Confusion Matrix:
[[963  2]
 [ 16 134]]

Classification Report:
              precision    recall  f1-score   support

    ham       0.98         1.00         0.99         965
    spam       0.99         0.89         0.94         150

 accuracy         0.98         0.95         0.98         1115
 macro avg       0.98         0.95         0.96         1115
 weighted avg    0.98         0.98         0.98         1115

Posterior Probability:
[[6.08367345e-05 9.99939163e-01]]
Predicted Class: spam
```

3. Information Theory

Aim

To calculate the entropy and information gain for all input attributes using the Play Tennis Dataset and identify the attribute with the highest information gain.

Program:

```
import pandas as pd
import math

from google.colab import files
uploaded = files.upload()

file_name = list(uploaded.keys())[0]
data = pd.read_csv(file_name)
print(data)

data = data.drop("Day", axis=1)
target = "Play"

def entropy(column):
    values = column.value_counts(normalize=True)
    ent = 0
    for p in values:
        ent -= p * math.log2(p)
    return ent

total_entropy = entropy(data[target])
print("\nTotal Entropy:", round(total_entropy, 4))

information_gain = {}
for attribute in data.columns[:-1]:
    weighted_entropy = 0
    for value in data[attribute].unique():
        subset = data[data[attribute] == value]
        weight = len(subset) / len(data)
        weighted_entropy += weight * entropy(subset[target])
    information_gain[attribute] = total_entropy - weighted_entropy
```

Output:

```
archive (9).zip(application/x-zip-compressed) - 21007 bytes, last modified: 7/20/2020 - 100% done
Saving archive (9).zip to archive (9).zip
  Day Outlook Temperature Humidity Wind Play
0   D1 Overcast      Mild Normal Strong Yes
1   D2 Sunny        Mild Normal Strong Yes
2   D3 NaN          Mild High Strong No
3   D4 Sunny        Mild High Weak Yes
4   D5 Sunny        Cool Normal Strong Yes
... ..
6661 D28 Sunny      Mild High Weak Yes
6662 D29 Rainy      Cool High Weak No
6663 D30 Overcast   Cool High Weak Yes
6664 D31 Sunny      Hot High Strong No
6665 D1 Sunny       Cool Normal Strong Yes

[6666 rows x 6 columns]

Total Entropy: 0.9635

Information Gain:
Outlook : 0.2322
Temperature : 0.2165
Humidity : 0.0912
Wind : 0.1036

Best Attribute: Outlook
```

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Dataset Preparation	20%	Correctly imports, explores, and prepares the dataset without errors.	Prepares the dataset with minor errors or omissions.	Dataset preparation is incomplete or contains several errors.	Fails to prepare or load the dataset correctly.
Implementation of Mathematical Concepts	30%	Correctly applies all required mathematical concepts (Probability, Bayes, Entropy, etc.) with accurate calculations.	Applies most concepts correctly with minor computational errors.	Applies concepts partially with noticeable calculation errors.	Unable to apply the required mathematical concepts correctly.
Tool/Code Execution	20%	Implements the experiment using appropriate tools (Python/Scikitlearn) with error-free execution.	Executes the experiment with minor coding or execution errors.	Code executes partially or requires significant corrections.	Unable to execute the experiment successfully.
Result Interpretation	20%	Correctly interprets the experimental results and relates them to the applied concept.	Interprets results with minor inaccuracies.	Interpretation is incomplete or partially correct.	Fails to interpret the experimental results.

Documentation & Presentation	10%	Experiment is well- documented with clear outputs, observations, and proper formatting.	Documentation is complete with minor presentation issues.	Documentatio n is incomplete or lacks clarity.	Poor or missing documentation and presentation.
---------------------------------	-----	--	---	---	--